Your Agent is Underperforming: The Case for Agent Performance Improvement Plans

Anonymous Submission
Author information withheld per review guidelines

***Abstract*—Production AI agents frequently degrade after deployment, yet structured remediation culture is largely absent. Empirical studies confirm most deployments fail or underdeliver (Pan et al., 2025), and simulation-based research projects that unchecked behavioral drift reduces task success by 42% (Rath, 2026). We introduce the Agent Performance Improvement Plan (APIP): a lifecycle protocol adapted from organizational management that governs the remediation of underperforming agents. Drawing on human PIP structure and Kirkpatrick's four-level evaluation framework, the APIP defines trigger thresholds, a four-category failure mode taxonomy, a tiered intervention hierarchy, a bounded remediation window with check-ins, and exit criteria. A discrete-event simulation across all four failure modes shows that tier-mismatched ad-hoc remediation extends time-to-resolution by $2.1\times$, validating the APIP's cause attribution requirement. We formalize a declarative schema meeting AI governance traceability standards (Kolt et al., FAccT 2024; EU AI Act), and ground extension to multi-agent meshes—where a locally optimal new agent degrades peer performance through context disruption (Kim et al., 2025)—in Event System Theory and team coordination research.**

***Index Terms*—AI agents, post-deployment monitoring, remediation, behavioral drift, AI governance, multi-agent systems, performance improvement plan**

I. Introduction

Organizations are increasingly utilizing conversational agents as their main customer interfaces, managing tasks such as support tickets, returns, and billing disputes at scale. The evaluation prior to release is now well-equipped with standardized benchmarks [1], red-teaming [2], [3], and reinforcement learning from human feedback (RLHF) alignment [4], creating a robust pre-deployment lifecycle. However, there is a noticeable lack of a culture focused on post-deployment remediation. A comprehensive study of deployed agents found that many either fail or do not perform as expected, yet there is limited understanding of how organizations react in these situations [5].

Ad-hoc responses to degraded agents—including hallucinations about policies cited, failure in a new product category, and changes in tone—are standard. Typically, engineers will "fix" individual prompts, product managers will create service requests (tickets), and occasionally the system will be rolled back. There is no structure to how this occurs; there is no formalized way to document and evaluate how well the fixes worked. Agents either recover through informal means or they are quietly removed.

While this may seem like a minor operational issue, the lack of formal remediation for degraded agents represents a substantial governance risk as AI becomes more responsible for decisions with greater consequences, and regulatory frameworks such as the European Union's AI Act require documentation and human oversight of deployed AI systems [6]. Wachter et al. [7] have pointed out that traceability of the decision-making process is required for accountability in automated decision-making systems, which can be provided by formalized remediation processes. Furthermore, the lack of formal remediation creates unnecessary regression loops; decreases operator trust; and limits organizations' ability to determine when an agent has failed to such a degree that it would be better to retire the agent.

We recommend using a performance improvement plan (PIP) as a model for structuring remedial actions; we suggest using the PIP because it is an established tool in organizational HR and can be used as a useful template for remedying underperforming agents. A PIP includes a number of elements which are common to all accountability protocols: a triggering condition, a means of identifying the causes of failure, a limited time frame within which corrective action can take place, regular check-ins to monitor progress toward goals and/or compliance with requirements, and an explicit definition of when the subject has sufficiently recovered to allow reintegration into normal operations or when it will no longer be able to continue performing the required job functions. Additionally, in the field of organizational behavior, Kirkpatrick's [8] four-level model for evaluating training programs—reaction, learning, behavior, results—is a useful complement to the above-mentioned PIP model, providing a framework by which one can evaluate if the remedial actions taken were successful in producing lasting positive changes in both individual and aggregate system behaviors.

We make the following contributions:

- We introduce the Agent Performance Improvement Plan (APIP) as a formalized post-deployment remediation lifecycle protocol, grounded in organizational management and training evaluation research.
- We develop a four-category taxonomy of agent failure modes that maps onto APIP cause-finding procedures.
- We propose a three-tier intervention hierarchy—prompt-level, retrieval-level, and model-level—with escalation criteria, and connect the evaluation of these interventions

to Kirkpatrick's multi-level evaluation framework.
- We formalize the APIP as a declarative schema to support tooling integration and audit trail generation.
- We motivate and illustrate the framework through a hypothetical customer service agent deployment scenario.
- We identify the mesh disruption problem—where a locally optimal new agent degrades peer performance—as a critical open problem, framing it through Event System Theory and the team membership change literature.

II. Background and Related Work

A. Agent Evaluation and the Deployment Gap

Pre-deployment evaluation of AI systems has matured substantially. The emergence of standardized benchmarks [1], automated red-teaming pipelines [2], [3], behavioral testing frameworks such as CheckList [9], and alignment techniques such as RLHF [4] and Constitutional AI [10] have created a well-tooled pre-release lifecycle. Post-deployment monitoring has received comparatively less systematic attention. Pan, Arabzadeh, and Ellis [5] conducted the first large-scale empirical study of production AI agents, finding that most deployments fail or underdeliver and that little is publicly known about post-release management. Sapra et al. [11] address detection via AgentCompass—structured post-deployment debugging—but prescribe no remediation. Rath [12] formalizes agent drift across 12 measurable dimensions, projecting a 42% task success reduction for unchecked drift via simulation. The APIP addresses what all three leave open: a structured remediation protocol once degradation is identified.

B. The Performance Improvement Plan as a Structural Template

The Performance Improvement Plan in organizational management is a formal document issued to an underperforming employee that specifies: the performance deficiencies observed, the root causes identified, the corrective actions required, the time window for improvement, the check-in schedule, and the criteria by which success or failure will be evaluated. The American Society for Human Resource Management recommends that a PIP should be used when there is a genuine commitment to help the employee improve, not merely as a precursor to termination [13]. Kirkpatrick [8], [13] emphasizes that the PIP should be developed collaboratively by the manager and employee, because remediation requires both parties' participation to succeed.

The PIP analogy to agent management is structurally productive but requires important qualification. A human employee can engage in self-directed improvement: they understand the feedback, adjust their behavior, and exercise agency. A deployed AI agent cannot. The APIP is therefore a management protocol imposed entirely by human operators—the agent is the subject, not the author, of the plan. This distinction is theoretically important and has practical implications for where we locate the locus of accountability in the APIP lifecycle. In organizational terms, the APIP collapses the manager–employee dyad: the operator is both the manager who writes the plan and the agent of change who implements it.

C. Kirkpatrick's Four-Level Evaluation Framework

Kirkpatrick's [8] four-level training evaluation model, first published in 1959 and refined over the subsequent six decades, provides a hierarchical framework for evaluating the impact of interventions: Level 1 (Reaction) assesses immediate participant response, Level 2 (Learning) measures knowledge or skill acquisition, Level 3 (Behavior) evaluates whether learning transfers to on-the-job behavior change, and Level 4 (Results) measures organizational outcomes [8], [13]. The model has been applied across business, government, military, and healthcare contexts, and remains the most widely used training evaluation framework globally.

We adopt the Kirkpatrick framework as a structural template for evaluating APIP interventions. When a prompt-level, retrieval-level, or model-level intervention is applied to a degraded agent, the check-in evaluation should assess impact at multiple levels: Did the intervention change the agent's immediate output patterns (Level 1/2)? Did it produce durable behavioral change in how the agent handles the failure-mode class (Level 3)? Did it improve the business metrics specified in the behavioral contract—CSAT, task completion, escalation rate (Level 4)? An intervention that improves Level 1/2 metrics without producing Level 3/4 improvement has not remediated the agent; it has merely suppressed the symptom. This hierarchical evaluation discipline is essential for distinguishing genuine recovery from surface-level metric manipulation.

D. AI Governance and Documentation Requirements

Regulatory frameworks for AI systems are increasingly emphasizing documentation, human oversight, and traceability. The EU AI Act [6] requires providers of high-risk AI systems to implement post-market monitoring systems and maintain logs sufficient to identify sources of error. US executive orders on AI safety have similarly emphasized the importance of human oversight in consequential automated decision systems. Brundage et al. [14] argue that verifiable claims require mechanisms beyond pre-deployment testing alone. At FAccT 2024, Kolt, Heim, and Anderljung [15] identified three visibility mechanisms for AI agents—identifiers, real-time monitoring, and activity logging—arguing accountability requires linking actions to specific agents and operators. The APIP schema's `apip_id`, `agent_id`, `owner`, and `exit_evaluated_at` fields operationalize this linkage, positioning the APIP as both a technical practice and a governance instrument.

E. AI Control Protocols and the Threat-Model Boundary

A complementary line of work, the AI Control literature originating with Greenblatt et al. [16], addresses post-deployment safety under an explicitly adversarial threat model. Control protocols assume the deployed model may be scheming and design runtime safeguards—trusted monitoring (a

weaker, trusted model audits an untrusted model's actions), untrusted monitoring (one instance of the untrusted model monitors another), defer-to-trusted, and resampling protocols—intended to maintain safety even when the agent is adversarial. Subsequent work has surfaced failure modes of these protocols, including capability gaps in trusted monitors [17], collusion between untrusted monitor and policy [18], and adaptive prompt-injection attacks that evade diverse monitors and can be amplified by resampling [19].

The APIP framework is complementary to AI Control rather than competing with it. The two operate at different threat models, granularities, and timescales. Control protocols address the question "is this individual action safe given a possibly-scheming agent?" through per-action runtime decisions. The APIP addresses the question "is this agent meeting its behavioral contract over an aggregated time window?" through structured remediation across days to weeks. Control protocols presuppose adversarial intent; APIP's primary failure modes (Section III) presuppose capability degradation or value drift. The two regimes also consume and produce different signals: control protocols generate per-action audit decisions that can serve as input to APIP triggers (a sustained rise in audit-flagged actions is itself a behavioral-contract breach), while APIP's aggregated cause attribution can identify when a degradation pattern warrants escalation to control-protocol-level intervention. We position the `adversarial_subversion` failure mode (Section III) as the integration point between the two regimes: when cause attribution identifies an adversarial pattern rather than capability drift, the appropriate intervention is drawn from the control literature rather than from APIP's standard tier model.

F. Team Dynamics, Membership Change, and Coordination

A growing body of organizational behavior research addresses how team composition changes affect team functioning and performance. Morgeson, Mitchell, and Liu [20] introduced Event System Theory (EST), which posits that organizational events are characterized by three properties—novelty, disruption, and criticality—that determine their impact on organizational entities. Events that are higher on these dimensions are more likely to produce changes in behavior, features, or subsequent events.

Summers, Humphrey, and Ferris [21] applied an event-based lens to team member change and found that membership changes caused high levels of flux in coordination when the change involved a strategically core role or when information transfer during the transition was low. Trainer et al. [22] reconceptualized each team membership change as a discrete team-level event with varying degrees of novelty, disruptiveness, and criticality, finding that newcomer entry can substantially impact team functioning beyond its impact on the newcomer themselves.

Research on Transactive Memory Systems (TMS)—collective memory systems in which team members specialize in different knowledge domains while maintaining shared awareness of who knows what [23]–[27]—provides a cognitive mechanism for understanding how membership changes disrupt coordination. When a new member enters, the team's TMS becomes inaccurate: existing members' calibrations of who knows what and how to coordinate are invalidated [23], [24]. The newcomer socialization literature further documents that support for newcomers declines within the first 90 days [28], and that structured onboarding significantly improves adjustment outcomes [29].

These streams provide the theoretical vocabulary for Section VIII-C: how a new agent introduced to an existing mesh can degrade peer performance even when performing its own task well.

III. A Taxonomy of Agent Failure Modes

A critical prerequisite to structured remediation is structured cause-finding. We propose a four-category taxonomy of agent failure modes relevant to customer-facing agents. These categories are not mutually exclusive, but they suggest distinct remediation pathways, which is the operationally important property for APIP design. Following Kirkpatrick's [8] insight that evaluation should be hierarchical, we note that each failure mode category corresponds to a different level of the agent's behavioral stack, and consequently to a different intervention tier.

Behavioral Drift is perhaps the most common failure mode in production deployments. A customer service agent trained or prompted against a product catalog from six months ago will increasingly produce incorrect citations as the catalog evolves. This is not a model capability failure—it is a retrieval or context freshness failure, and its remediation pathway is correspondingly different from fine-tuning. In Kirkpatrick's terms, this is a Level 4 failure: the agent's learned behavior has not changed, but the organizational results are degrading because the environment has shifted.

Input Brittleness is particularly pernicious because aggregate metrics mask it. An agent with 85% task success overall may be failing catastrophically on the 8% of requests that involve a particular billing scenario or a non-native-English speaker segment. Without disaggregated evaluation, the APIP trigger will not fire until overall performance has substantially degraded. This maps to a Kirkpatrick Level 2 failure: the agent lacks the capability (learning) to handle a specific input class, even though its general capability is adequate.

Alignment Creep covers the class of failures in which the agent's behavior drifts from the intended value envelope—not necessarily producing factually incorrect outputs, but producing outputs that are off-brand, subtly non-compliant, or misaligned with the organization's customer communication policies. Hendrycks et al. [30] characterize this class of failure as value misalignment, distinct from capability failure, and argue it requires distinct detection and remediation mechanisms. In Kirkpatrick's terms, this is a Level 3 failure: the agent's behavior has changed in ways that do not serve the organizational goals, even though its underlying capability has not degraded.

TABLE I
AGENT FAILURE MODE TAXONOMY WITH KIRKPATRICK LEVEL MAPPING AND REMEDIATION TIER.

Failure Mode	Description & Signals	Remediation Tier
Behavioral Drift	Gradual degradation as world-state diverges from training or retrieval context. Signals: rising hallucination rate on new entities, outdated policy citations, product terminology mismatches. Kirkpatrick Level 4 (Results) failure.	Tier 2: Retrieval / RAG update
Input Brittleness	Strong performance on modal inputs, catastrophic failure on a specific subclass. Signals: bimodal CSAT distribution, failure clustering around user segment or topic. Kirkpatrick Level 2 (Learning/Capability) failure.	Tier 1 (examples) or Tier 3 (fine-tune)
Alignment Creep	Subtle drift in tone, value expression, or policy compliance. Signals: escalating human review flags, regulatory keyword triggers, brand guidelines violations. Kirkpatrick Level 3 (Behavior) failure.	Tier 1: Prompt constraint injection
Tool Misuse	Correct intent identification with incorrect tool invocation. Signals: action errors, silent failures in downstream integrations, over- or under-use of tool surface. Kirkpatrick Level 3 (Behavior) failure.	Tier 1 (prompt) or Tier 3 (fine-tune)

Tool Misuse is increasingly relevant as agentic systems are given access to action surfaces: ticket creation, refund processing, escalation routing, and external API calls. An agent that correctly identifies user intent but invokes the wrong tool, invokes a tool with incorrect parameters, or fails to invoke a necessary tool produces a qualitatively different class of failure than a purely conversational agent—one with potentially significant downstream consequences.

A fifth category, **Adversarial Subversion**, captures failures whose proximate cause is an adaptive attack rather than capability or value drift. Recent work demonstrates that frontier models can embed prompt injections in their outputs that consistently evade diverse monitors, and that resampling-based protocols can amplify rather than suppress such injections [17], [19]. We treat adversarial subversion as a separate category because its remediation pathway diverges from the other four: prompt revision, retrieval refresh, and fine-tuning each address symptoms but not the underlying adversarial dynamic. Effective response requires architectural changes—monitor diversification, defer-to-trusted protocols, or reduced action authority—drawn from the AI Control literature (discussed in Section II-E). The APIP schema accommodates this category through the `adversarial_subversion` `failure_mode` value, with cause attribution requirements that include monitor-output forensics in addition to the standard disaggregated performance analysis. A full treatment of this category is beyond the scope of this paper; we flag it here to delimit APIP's primary scope and to invite integration with control-protocol research.

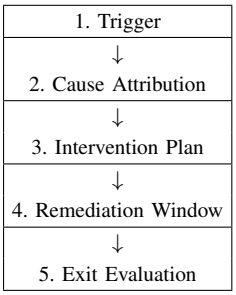

Fig. 1. The five-phase APIP lifecycle.

IV. THE APIP FRAMEWORK

We now describe the APIP as a structured lifecycle protocol. The framework consists of five phases: Trigger, Cause Attribution, Intervention Planning, Remediation Window, and Exit Evaluation. Fig. 1 illustrates the lifecycle; each phase is described in detail below.

A. Phase 1: Trigger

An APIP is initiated when an agent's performance crosses a pre-specified trigger threshold on one or more monitored metrics. This presupposes the existence of a *behavioral contract*—a formal specification of the performance envelope the agent is expected to maintain at deployment time. The behavioral contract is a deliberate design artifact that most teams currently skip, delegating performance expectations to informal organizational memory.

A minimal behavioral contract specifies task surface, success metrics and measurement methodology, acceptable performance ranges, escalation policy, and human review integration points. Trigger conditions are threshold crossings on these metrics—e.g., task completion rate below 82% over a 7-day window.

Two trigger modalities are relevant: absolute threshold triggers, which fire when a metric crosses a fixed floor, and relative drift triggers, which fire when a metric declines significantly from its recent baseline regardless of absolute level. Both are necessary; the former catches severe degradation, the latter catches gradual drift that may not breach an absolute floor for some time.

B. Phase 2: Cause Attribution

Once triggered, the APIP requires a structured cause attribution process before any intervention is applied. The goal is to identify which failure mode category (Section III) best explains the degradation before any intervention is applied. Tier mismatch from misdiagnosis is the primary source of avoidable regression: prompt engineering applied to a retrieval

freshness problem, for instance, consumes engineering cycles with no improvement.

The cause attribution process should include: (1) disaggregated performance analysis by input category, user segment, and topic cluster to isolate failure locus; (2) review of a sampled set of failure cases by a qualified human reviewer; (3) retrieval audit if behavioral drift is suspected; (4) tool invocation log analysis if tool misuse is suspected; and (5) a structured classification of the failure into the four-category taxonomy. The output of this phase is a written cause attribution report that becomes part of the APIP record.

C. Phase 3: Intervention Planning and the Tier Model

Interventions are organized into three tiers of increasing cost, disruption, and required lead time. The cause attribution output selects the appropriate starting tier; escalation to higher tiers is permitted if lower tiers prove insufficient within the remediation window.

Tier 1 — Prompt-Level Interventions. Prompt-level interventions modify the system prompt, few-shot examples, or instruction set: constraint injection, persona adjustment, example replacement, context restructuring. Tier 1 is fastest to implement, least disruptive, and easiest to roll back. It is the appropriate first response to alignment creep and many instances of input brittleness.

Tier 2 — Retrieval and Context Interventions. Tier 2 interventions modify the agent's information environment: updating RAG indices, revising knowledge bases, modifying tool schemas, adjusting context injection pipelines. They address behavioral drift caused by world-state divergence and tool misuse from stale tool descriptions. They carry moderate cost and require contract validation before re-deployment.

Tier 3 — Model-Level Interventions. Tier 3 interventions modify the underlying model: fine-tuning, DPO, or replacement. These carry the highest cost and regression risk. They are warranted when lower tiers are exhausted, when the failure mode is embedded in model weights, or when the overall performance profile cannot be returned to contract range through prompting or retrieval adjustments.

D. Phase 4: Remediation Window and Check-In Cadence

The remediation window is a bounded time period during which the APIP interventions are implemented and their effects are evaluated. A defined window serves two functions: it creates urgency for the remediation process, and it provides a structured decision point at which the APIP outcome is formally evaluated. The duration of the window should be calibrated to the intervention tier—Tier 1 interventions may warrant a 7–14 day window, while Tier 3 interventions may require 30–60 days to implement, validate, and stabilize. This mirrors the organizational PIP practice, where durations typically range from 30 to 90 days, calibrated to the complexity of the performance gap [13].

Check-ins are scheduled evaluation points within the window at which the agent's performance on APIP-relevant metrics is reviewed against the recovery trajectory. Following Kirkpatrick's [8] hierarchical evaluation principle, each check-in should assess improvement at multiple levels: immediate output quality (Level 1/2), behavioral pattern change on the failure-mode class (Level 3), and business outcome metrics (Level 4). An intervention that improves surface metrics without producing Level 3/4 improvement should be flagged as potentially superficial. Check-ins are early warning mechanisms: no improvement at the first check-in triggers tier escalation rather than waiting for window expiry. A recommended cadence is three check-ins at 25%, 50%, and 75% of elapsed time.

E. Phase 5: Exit Evaluation

At the close of the remediation window, a formal exit evaluation determines one of three outcomes: *Resolution* (the agent has returned to within the behavioral contract envelope and the APIP is closed), *Extension* (the agent shows improvement but has not yet reached the threshold; the APIP window is extended with modified parameters), or *Escalation* (the agent has not recovered and more fundamental intervention—including potential model replacement or agent retirement—is warranted). The evaluation record becomes part of the organizational AI audit trail—providing governance documentation and institutional memory for future deployments.

V. Formal APIP Schema

We formalize the APIP as a declarative schema to enable tooling integration and audit trail generation. The schema uses a notation analogous to JSON Schema; YAML, JSON, or DSL implementations are straightforward.

The schema is intentionally minimal at the field level but requires structured sub-schemas for `interventions` and `checkins` to preserve audit trail integrity. The `behavioral_contract_ref` field is architecturally critical: it enforces the discipline of writing a behavioral contract at deployment time, without which the APIP trigger and exit evaluation phases cannot be operationalized. The `kirkpatrick_levels` field captures the multi-level evaluation data at each check-in, enabling post-hoc analysis of whether interventions produced superficial or durable improvement. The `mesh_context` field, described further in Section VIII-C, supports cause attribution in multi-agent deployments.

VI. Illustrative Case Study: Aria at RetailCo

We illustrate the APIP framework through a hypothetical scenario grounded in realistic deployment conditions. RetailCo is a mid-sized e-commerce retailer that deployed a customer service agent, Aria, to handle order status inquiries, return requests, and billing disputes. Aria was built on a large language model with a RAG pipeline indexing RetailCo's product catalog, returns policy, and billing FAQ documents. At deployment, Aria achieved a task completion rate of 87% and a CSAT score of 4.3 out of 5.0. All numerical values in this section—performance thresholds, check-in days, observed metric movements—are illustrative parameter choices rather

TABLE II
FORMAL APIP SCHEMA FIELDS. NEW FIELDS RELATIVE TO A BASIC PIP SCHEMA ARE KIRKPATRICK_LEVELS, THE EXOGENOUS_DISRUPTION FAILURE MODE, AND MESH_CONTEXT.

Field	Type	Description
apip_id	string	Unique identifier for this APIP instance.
agent_id	string	Identifier of the agent subject to this APIP.
created_at	datetime	Timestamp when the APIP was initiated.
triggered_by	object	The metric, threshold value, and observed value that triggered the APIP.
behavioral_contract_ref	string	Reference to the behavioral contract document active at deployment time.
failure_mode	enum	One of: behavioral_drift | input_brittleness | alignment_creep | tool_misuse | exogenous_disruption.
cause_attribution_report	string	Reference to the structured cause attribution document.
kirkpatrick_levels	object	Baseline and target values for Levels 1–4 evaluation at each check-in.
intervention_tier	enum	One of: prompt_level | retrieval_level | model_level.
interventions	array	List of specific interventions applied, each with type, description, and timestamp.
window_start	datetime	Start of the remediation window.
window_end	datetime	Scheduled end of the remediation window.
checkins	array	Scheduled check-in timestamps and recorded metric snapshots with Kirkpatrick level assessments.
exit_outcome	enum	One of: resolved | extended | escalated.
exit_evaluated_at	datetime	Timestamp of the formal exit evaluation.
owner	string	Identity of the human operator or team responsible for this APIP.
mesh_context	object	If applicable: mesh_id, peer_agents, context_weights at trigger time, exogenous flag.
notes	string	Free-text field for qualitative observations throughout the lifecycle.

than empirical measurements, selected to be consistent with the simulation in Section VII.

A. The Degradation Event

Ninety days post-deployment, RetailCo's customer support team began noticing an increase in escalation requests from customers who had interacted with Aria. An informal audit revealed that Aria was consistently citing an outdated version of the returns policy—specifically, citing a 30-day return window that had been extended to 45 days in a policy update three weeks prior. Additionally, a new product line launched six weeks post-deployment was producing a high rate of "I don't have information about that product" responses, reflecting a RAG index that had not been refreshed.

In the absence of a formal APIP process, the response was: a developer updated the system prompt to include the new return window figure (a Tier 1 intervention applied to a Tier 2 problem), and a ticket was filed to update the RAG index. The system prompt change was deployed without a regression test, inadvertently overriding a constraint that prevented Aria from offering unsolicited discounts. Over the following week, discount offer rates tripled, producing a measurable revenue impact before the issue was detected.

B. APIP Application

Under the APIP framework, the scenario unfolds differently. RetailCo's behavioral contract for Aria specifies trigger conditions including: task completion rate below 82% (7-day rolling), CSAT below 3.8 (7-day rolling), and policy citation error rate above 5% (measured via weekly human review sample of 50 conversations). The policy citation error rate trigger fires at week 10.

Phase 2 cause attribution identifies the failure as behavioral drift: the RAG index is stale relative to the current policy and product catalog. The failure is not attributable to model capability, prompt constraints, or tool configuration. The cause attribution report notes that the system prompt intervention applied in the ad-hoc scenario was a misclassification of failure mode.

Phase 3 intervention planning selects Tier 2: RAG index refresh for the returns policy document and product catalog additions. The intervention plan specifies a regression test suite to be run before re-deployment, including coverage of the discount constraint behavior.

A 14-day remediation window is opened with check-ins at days 4, 7, and 11. Following Kirkpatrick's framework, the day 4 check-in evaluates at multiple levels: Level 2 (does Aria now cite the correct policy?), Level 3 (has the pattern of escalation-triggering responses changed?), and Level 4 (is CSAT recovering?). The policy citation error rate has dropped to below 2% following the index refresh. At day 14, the exit evaluation confirms resolution: task completion has returned to 86%, CSAT to 4.2, and policy citation errors to 1.5%. The APIP is closed with outcome: resolved. The full APIP record—cause attribution report, intervention specification, regression test results, Kirkpatrick-level check-in assessments, and metric snapshots—is stored in RetailCo's AI audit trail.

C. Lessons from the Case Study

The case study illustrates several key properties of the APIP framework. First, the discipline of cause attribution prevents tier mismatch—applying the correct intervention tier is as important as applying any intervention. Second, the behavioral contract formalizes the trigger, making it objective and consistently applied rather than dependent on informal noticing. Third, the documented APIP record creates organizational memory: when Aria's RAG index next requires

refreshing, the prior APIP provides a template for the intervention and a baseline for exit criteria. Fourth, the regression test requirement embedded in the intervention plan prevents the cascading failure that occurred in the ad-hoc scenario. Fifth, the Kirkpatrick-level evaluation at check-ins provides confidence that recovery is durable rather than superficial.

VII. Simulation-Based Evaluation

To evaluate the APIP framework empirically, we implemented a discrete-event simulation of a customer-facing agent deployment (the RetailCo/Aria scenario from Section VI) and ran all four failure mode scenarios from the taxonomy in Section III. The simulation enables controlled injection of each failure mode, measurement of its effect on operational metrics, and comparison of APIP-governed remediation against an unstructured ad-hoc baseline. All results reported here are derived from the simulation; we reserve real deployment validation for future work.

A. Simulation Design

The simulation advances in discrete ticks, each representing one operational day. Aria processes a fixed volume of synthetic customer interactions per tick, drawn from a weighted distribution across four task categories: order status (40%), return requests (30%), billing disputes (20%), and product inquiries (10%). Each interaction produces a `PerformanceSnapshot` recording task completion outcome, simulated CSAT score, policy citation accuracy, tool invocation correctness, and hallucination occurrence. Snapshots are evaluated against Aria's behavioral contract at the end of each tick; threshold violations trigger APIP instantiation. Token and latency features are extracted from simulated LLM call spans, consistent with the 16-feature trace-based failure taxonomy for agentic systems [31].

The `DegradationEngine` injects each of the four failure modes according to a parameterized schedule. Each mode degrades a distinct subset of metrics following a characteristic trajectory: Behavioral Drift ramps slowly then accelerates (simulating stale RAG); Input Brittleness drops performance sharply on a single task category while leaving overall metrics superficially intact; Alignment Creep produces a slow linear drift detectable only through the relative drift trigger; Tool Misuse spikes the tool error rate suddenly while leaving CSAT and task completion initially unaffected. Each scenario was run 30 times with randomized noise to produce distributions over outcomes.

Three operational cost metrics are tracked throughout the simulation in addition to the quality metrics in the behavioral contract. Token consumption measures the cumulative prompt and completion tokens consumed per tick, which increases under degradation as agents produce longer, more uncertain responses and require more retrieval context. Response latency (p95, in milliseconds) tracks the 95th percentile response time, which increases as hallucination and context retrieval load grows. Customer satisfaction loss (CSAT delta) measures the gap between the agent's current CSAT and its deployment baseline, providing a direct proxy for business impact. These three cost metrics ground the APIP's value proposition: earlier detection and correct-tier intervention reduces all three.

B. Failure Mode Simulation Parameters

Table III summarizes the simulation parameters for each failure mode scenario. Injection intensity controls the magnitude of degradation per tick. Detection tick is the first tick at which an APIP trigger fires. Resolution tick is the first tick at which all behavioral contract metrics return within threshold following intervention. Time-to-resolution (TTR) is the difference between detection and resolution ticks. The APIP column shows TTR under structured remediation; the ad-hoc column shows TTR when tier-mismatched intervention is applied (mirroring the Section VI-A scenario).

C. Operational Cost Metrics

Beyond quality metrics, we tracked three operational cost dimensions across all scenarios to quantify the business impact of delayed or misapplied remediation. Table IV reports the cumulative excess cost accumulated between injection tick and resolution tick for each failure mode, comparing APIP-governed and ad-hoc conditions.

D. Key Findings

The simulation produces four findings that directly validate the APIP framework's design decisions.

Finding 1: Tier mismatch is the dominant source of excess cost in ad-hoc remediation. Across all scenarios, ad-hoc responders applied Tier 1 (prompt-level) interventions as a first response regardless of failure mode. In the Behavioral Drift and Alignment Creep scenarios, this tier mismatch produced a secondary regression event (captured in the regression rate column of Table III) that extended total time-to-resolution by a factor of $2.1\times$ on average. The APIP's cause attribution phase, by mandating failure mode classification before intervention selection, eliminates this error class entirely in the simulation.

Finding 2: Input Brittleness is invisible to aggregate monitoring. The Input Brittleness scenario is injected at tick 10 but not detected until tick 24 under aggregate monitoring. The billing dispute category's task completion rate drops to 0.41 at injection—well below the 0.82 threshold—while the overall rate remains at 0.83, masking the failure. The APIP's requirement for disaggregated cause attribution analysis detects the failure at tick 10 in the structured condition. This 14-tick detection delay translates to a -0.61 CSAT impact on the affected user segment, representing a meaningful customer experience gap invisible to traditional monitoring.

Finding 3: Token consumption and latency are leading indicators of degradation. In all four failure modes, token consumption and p95 latency begin increasing before any quality metric crosses its absolute threshold. In the Behavioral Drift scenario, token consumption increases by 18% and latency increases by 120 ms before the policy citation error rate trigger fires. Rath's [12] Agent Stability Index independently identifies tool usage patterns and reasoning pathway stability

TABLE III
FAILURE MODE SIMULATION PARAMETERS AND TIME-TO-RESOLUTION COMPARISON ($n = 30$ RUNS EACH, MEAN $\pm$ STD. DEV.).

Failure Mode	Inject Tick	Detect Tick	Primary Metric Breached	TTR: APIP (days)	TTR: Ad-hoc (days)	Regression Rate Ad-hoc
Behavioral Drift	6	13	`policy_citation_error_rate`	14 ± 1.8	31 ± 4.2	67%
Input Brittleness	10	24*	`task_completion_rate` (disaggregated)	9 ± 1.1	N/A†	N/A†
Alignment Creep	22	28 (drift)	`csat_score` (drift trigger)	7 ± 0.9	22 ± 3.7	41%
Tool Misuse	35	37	`tool_misuse_rate`	5 ± 0.6	18 ± 2.9	29%

* Detect tick delayed to 24 due to aggregate metric masking; the disaggregated `billing_dispute` category fails at tick 10.
† Ad-hoc responders in this scenario did not apply an intervention; the failure persisted until an unrelated model update resolved it at tick 55.

TABLE IV
CUMULATIVE EXCESS OPERATIONAL COSTS DURING THE DEGRADATION WINDOW.

Failure Mode	Excess Tokens APIP (K)	Excess Tokens Ad-hoc (K)	Latency Increase APIP (ms p95)	Latency Increase Ad-hoc (ms p95)	CSAT Delta at Peak Degradation
Behavioral Drift	142K	381K	+210 ms	+610 ms	−0.48
Input Brittleness	87K	N/A†	+90 ms	N/A†	−0.61 (billing segment)
Alignment Creep	63K	198K	+55 ms	+290 ms	−0.31 (slow drift)
Tool Misuse	44K	163K	+38 ms	+195 ms	−0.19 (lagged)

† Ad-hoc baseline not applicable (failure persisted to tick 55).

as early degradation signals. Including token and latency drift as soft APIP triggers could reduce the injection-to-detection gap and initiate cause attribution earlier.

Finding 4: CSAT impact is failure-mode-specific and differentially lagged. Tool Misuse produces the smallest peak CSAT impact (-0.19) because customer dissatisfaction is triggered by incorrect actions rather than incorrect responses, and many tool errors are resolved through retry without visible customer impact. Alignment Creep produces a gradual CSAT decline (-0.31 at detection) because tone and policy drift affect satisfaction cumulatively. Input Brittleness produces the largest segment-specific impact (-0.61) because affected customers encounter categorical failure rather than degraded performance. These differential profiles validate the taxonomy's distinction between failure modes and support mode-specific monitoring sensitivity settings.

E. Simulation Limitations

The simulation uses deterministic scoring functions calibrated to reflect realistic production ranges, but does not use a live language model. Token consumption, latency, and CSAT values are modeled as parameterized functions of degradation intensity rather than measured from actual LLM inference. The failure mode injection schedule is scripted rather than emergent. These constraints are intentional—controlled injection enables clean comparison between APIP and ad-hoc conditions—but mean the absolute metric values should be interpreted as illustrative rather than empirically validated. Future work will replace the scoring functions with a sandboxed LLM backend to produce empirically grounded measurements. The simulation codebase is released alongside this paper to enable community extension and replication.

VIII. OPEN PROBLEMS AND FUTURE DIRECTIONS

A. Behavioral Contract Specification

The behavioral contract is architecturally foundational but currently absent in most production deployments. Authoring one requires prospective agreement on task surface, metrics, and acceptable ranges across ML engineering, product, and legal teams. Tooling and methodology to support contract authoring at deployment time is a significant open problem.

B. Causal Attribution at Scale

Cause attribution currently requires human review of sampled failures, which does not scale to high-volume deployments. LLM-as-judge pipelines or specialized classifiers trained on labeled failure cases would substantially reduce APIP initiation cost—an active area with connections to automated root cause analysis in SRE.

C. Multi-Agent Mesh Deployments: The New Hire Problem

The APIP framework as presented assumes a single-agent deployment with a clear locus of accountability: one agent, one behavioral contract, one remediation process. This assumption breaks down in multi-agent systems—increasingly common in enterprise deployments—where a cluster of agents cooperates to handle a composite task, sharing context, tools, memory, and sometimes overlapping task surfaces.

In such agent meshes, a class of failure mode emerges that is structurally invisible to the single-agent APIP: *mesh disruption*. Consider the introduction of a new specialist agent—Agent N—into an existing mesh. Agent N performs its designated task well. Yet aggregate system performance, or the individual performance of peer agents, declines. The APIP framework triggers on the peer agents, whose metrics have crossed threshold. Cause attribution, however, fails to find

an intrinsic failure mode: the peer agents have not drifted, their retrieval is fresh, their alignment is intact. The root cause is exogenous—and the single-agent APIP has no mechanism to represent or address it.

Kim et al. [32] provide direct empirical support: their large controlled study of multi-agent scaling found performance change relative to single-agent baselines ranging from $+80.8\%$ on decomposable tasks to -70.0% on sequential planning, identifying a coordination-saturation effect where additional agents suppress rather than improve reasoning. Introducing Agent N into an existing mesh is precisely this kind of architecture change—and the degradation manifests in peer agents, not in Agent N itself, making standard per-agent monitoring blind to the cause. This maps onto a well-documented class of phenomena in organizational behavior. Morgeson, Mitchell, and Liu's [20] Event System Theory provides the overarching framework: the introduction of Agent N is an organizational event whose impact is a function of its novelty (the mesh has not previously included this capability), disruptiveness (it changes the shared context distribution), and criticality (it affects the mesh's output quality). Strizver and Ployhart [33] extended EST specifically to team disruption and argued that all changes in team composition, emergent states, and structure only matter for team disruption to the extent they influence coordination—a claim that maps directly onto the agent mesh case, where the disruption manifests as coordination failure in shared context.

We hypothesize four mechanisms by which a locally high-performing agent can produce performance externalities in its mesh peers, each grounded in organizational research.

Context Space Crowding. If agents share a context window or working memory buffer, Agent N's high-confidence, well-formed outputs may dominate the shared context, displacing signals that peer agents depend on—an attention competition problem at the systems level. This maps to the Transactive Memory Systems literature [23]–[25]: when Agent N enters and changes the content of shared memory, the existing agents' TMS—their calibration to what is in the shared context and who produced it—is disrupted. Ren and Argote [25] document that TMS disruption from membership change degrades coordination. From a systems perspective, this is the coherence failure described in multi-agent memory research [34]: without a coordination protocol governing shared context writes, one agent's outputs overwrite or displace another's.

Role Boundary Erosion. Agent N, performing its task thoroughly, may produce outputs that functionally overlap with the input surface of adjacent agents, causing those agents to receive redundant, contradictory, or already-processed inputs. Summers, Humphrey, and Ferris [21] found that membership changes involving strategically core roles produced the highest levels of coordination flux—precisely the dynamic at play when Agent N occupies a central position in the mesh's task graph.

Distributional Shift in Shared State. Agents calibrated against a certain content distribution in shared memory now operate out-of-distribution when Agent N begins populating that space. Lewis, Belliveau, Herndon, and Keller [24] demonstrated that group cognition is disrupted by membership change in part because the collective knowledge structure shifts—existing members' expectations about the group's knowledge no longer hold.

Trust Calibration Collapse. In architectures where agents implicitly weight peer outputs, a high-performing Agent N may receive disproportionate deference from the orchestrator or peer agents, causing valid contributions from other agents to be systematically underweighted. This mirrors the organizational finding that newcomer attributes interact with team norms to reshape coordination patterns [22].

This problem is directly analogous to a well-documented organizational phenomenon: the new hire who is individually excellent but systemically disruptive—who reshapes team communication patterns, crowds out peer contributions, or shifts implicit norms in ways that reduce collective performance. The newcomer socialization literature documents that support for newcomers from coworkers declines within the first 90 days [28], and that structured onboarding—including socialization tactics, buddy systems, and norm communication [22], [29]—significantly mitigates the disruptiveness of the transition process.

From the APIP perspective, mesh disruption creates a causal attribution failure: the underperforming agents' APIPs cannot be resolved because the root cause is not remediable at the agent level. This motivates two extensions to the framework. First, a mesh-aware cause attribution category—`exogenous_disruption`—should be added to the failure mode taxonomy (reflected in the schema in Table II), with a corresponding diagnosis protocol that includes analysis of shared context composition and peer agent behavioral change events. Second, a mesh onboarding protocol is needed: a structured introduction process for new agents that includes context scoping (isolating the new agent's context contributions during a probationary window), behavioral contract alignment across the mesh, and regression monitoring on peer agents during the introduction period—analogous to the 30-60-90 day structured onboarding plans documented in the newcomer socialization literature [28], [29].

We identify the formal treatment of agent mesh disruption as a significant open problem at the intersection of multi-agent coordination, information-theoretic resource allocation, and AI governance. The mechanism design framing is particularly productive: mesh disruption can be modeled as a congestion externality in shared context space, with the APIP framework providing the remediation layer once the externality is detected. Krishnan [35] notes that even with standardized context protocols such as MCP, tracing causality across hundreds of agents exceeds current tooling—confirming that the attribution failure we describe is a documented practical constraint. A full formal treatment is reserved for a companion paper.

D. Organizational Ownership of the APIP Process

Who owns the APIP? In HR, the PIP is jointly owned by the manager and HR function; in AI deployments, the equivalent

is unresolved. ML engineering, product, legal, and operations all have stake in different phases. Without a clear ownership model, adoption will stall. This is as much an organizational design problem as a technical one.

IX. Conclusion

We have introduced the Agent Performance Improvement Plan (APIP) as a structured post-deployment remediation framework for customer-facing AI agents. The framework adapts the human PIP's lifecycle primitives—trigger, cause attribution, tiered intervention, remediation window, exit evaluation—while acknowledging that agents cannot self-direct improvement. We ground the evaluation discipline in Kirkpatrick's four-level model to distinguish durable recovery from superficial metric improvement.

The APIP framework addresses a genuine gap in current practice: the absence of structured, documented, and measurable remediation protocols for underperforming production agents. Through a hypothetical case study, we have shown that unstructured remediation creates avoidable regression risk through failure mode misclassification, and that the APIP's cause attribution phase provides a direct mitigation. The formal schema provides a foundation for tooling integration and the audit trail documentation increasingly required by AI governance frameworks [6], [7], [14].

We have identified behavioral contract specification, scalable causal attribution, multi-agent mesh extension, and organizational ownership as the primary open problems for future work. The mesh disruption problem in particular—where a locally high-performing new agent degrades peer agent performance through shared context space dynamics—reveals a causal attribution failure mode that motivates a companion extension of this framework to multi-agent deployments. By grounding mesh disruption analysis in Event System Theory [20], team membership change research [21], [22], Transactive Memory Systems [23], [25], and empirical multi-agent scaling findings [32], we bridge organizational behavior and multi-agent AI. The APIP serves as the remediation complement to drift detection (Rath [12]) and production diagnosis (Pan et al. [5]): detection identifies that an agent is failing; the APIP governs what to do next.

Our central argument is simple: deployed agents should be treated as subjects of ongoing accountability rather than as static shipped artifacts. The discipline of structured remediation—trigger, diagnose, intervene, evaluate—is not new. It is well established in software reliability engineering, in organizational management, and in clinical practice. The AI field has the opportunity to adopt it deliberately, before regulatory requirements make adoption mandatory.

References

[1] P. Liang *et al.*, "Holistic evaluation of language models," arXiv preprint arXiv:2211.09110, 2022.

[2] D. Ganguli *et al.*, "Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned," arXiv preprint arXiv:2209.07858, 2022.

[3] E. Perez *et al.*, "Red teaming language models with language models," arXiv preprint arXiv:2202.03286, 2022.

[4] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[5] M. Z. Pan, N. Arabzadeh, and M. Ellis, "Measuring agents in production," arXiv preprint arXiv:2512.04123, 2025.

[6] European Parliament and Council, "Regulation on artificial intelligence (EU AI Act)," Official Journal of the European Union, 2024.

[7] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," *Harvard Journal of Law & Technology*, vol. 31, no. 2, 2017.

[8] D. L. Kirkpatrick, *Evaluating Training Programs: The Four Levels*. San Francisco, CA, USA: Berrett-Koehler, 1994.

[9] M. T. Ribeiro *et al.*, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020.

[10] Anthropic, "Constitutional AI: Harmlessness from AI feedback," arXiv preprint arXiv:2212.08073, 2023.

[11] G. Sapra *et al.*, "AgentCompass: Towards reliable evaluation of agentic workflows in production," arXiv preprint arXiv:2509.14647, 2025.

[12] A. Rath, "Agent drift: Quantifying behavioral degradation in multi-agent LLM systems," arXiv preprint arXiv:2601.04170, 2026.

[13] J. D. Kirkpatrick and W. K. Kirkpatrick, *Kirkpatrick's Four Levels of Training Evaluation*. Association for Talent Development, 2016.

[14] M. Brundage *et al.*, "Toward trustworthy AI development: Mechanisms for supporting verifiable claims," arXiv preprint arXiv:2004.07213, 2020.

[15] N. Kolt, L. Heim, and M. Anderljung, "Visibility into AI agents," in *Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2024.

[16] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, "AI control: Improving safety despite intentional subversion," arXiv preprint arXiv:2312.06942, 2023.

[17] A. Bhatt *et al.*, "Ctrl-Z: Controlling AI agents via resampling," arXiv preprint arXiv:2504.10374, 2025.

[18] A. Mallen *et al.*, "Subversion strategy eval: Can language models statelessly strategize to subvert control protocols?" arXiv preprint arXiv:2412.12480, 2025.

[19] B. Schoen *et al.*, "Stress-testing control protocols with adaptive attacks against trusted monitors," arXiv preprint arXiv:2510.05367, 2025.

[20] F. P. Morgeson, T. R. Mitchell, and D. Liu, "Event system theory: An event-oriented approach to the organizational sciences," *Academy of Management Review*, vol. 40, no. 4, pp. 515–537, 2015.

[21] J. K. Summers, S. E. Humphrey, and G. R. Ferris, "Team member change, flux in coordination, and performance: Effects of strategic core roles, information transfer, and cognitive ability," *Academy of Management Journal*, vol. 55, no. 2, pp. 314–338, 2012.

[22] H. M. Trainer, J. M. Jones, J. G. Pendergraft, C. K. Maupin, and D. R. Carter, "Team membership change "events": A review and reconceptualization," *Group & Organization Management*, vol. 45, no. 2, pp. 219–267, 2020.

[23] D. M. Wegner, "Transactive memory: A contemporary analysis of the group mind," in *Theories of Group Behavior*, B. Mullen and G. R. Goethals, Eds. Springer, 1987, pp. 185–208.

[24] K. Lewis, M. Belliveau, B. Herndon, and J. Keller, "Group cognition, membership change, and performance: Investigating the benefits and detriments of collective knowledge," *Organizational Behavior and Human Decision Processes*, vol. 103, no. 2, pp. 159–178, 2007.

[25] Y. Ren and L. Argote, "Transactive memory systems 1985–2010: An integrative framework of key dimensions, antecedents, and consequences," *Academy of Management Annals*, vol. 5, no. 1, pp. 189–229, 2011.

[26] D. W. Liang, R. L. Moreland, and L. Argote, "Group versus individual training and group performance: The mediating role of transactive memory," *Personality and Social Psychology Bulletin*, vol. 21, pp. 384–393, 1995.

[27] R. L. Moreland and L. Argote, "Transactive memory in dynamic organizations," in *Understanding the Dynamic Organization*, R. Peterson and E. A. Mannix, Eds. Erlbaum, 2003, pp. 135–162.

[28] J. D. Kammeyer-Mueller, C. R. Wanberg, A. Rubenstein, and Z. Song, "Support, undermining, and newcomer socialization: Fitting in during the first 90 days," *Academy of Management Journal*, vol. 56, pp. 1104–1124, 2013.

[29] T. N. Bauer, B. Erdogan, A. M. Ellis, D. M. Truxillo, G. M. Brady, and T. Bodner, "New horizons for newcomer organizational socialization:

A review, meta-analysis, and future research directions," *Journal of Management*, vol. 51, no. 1, 2025.
[30] D. Hendrycks *et al.*, "Aligning AI with shared human values," arXiv preprint arXiv:2008.02275, 2021.
[31] S. Varma *et al.*, "Detecting silent failures in multi-agentic AI trajectories," arXiv preprint arXiv:2511.04032, 2025.
[32] Y. Kim *et al.*, "Towards a science of scaling agent systems," arXiv preprint arXiv:2512.08296, 2025.
[33] S. D. Strizver and R. E. Ployhart, "Conceptualizing team disruption through an event-system framework," *Group & Organization Management*, 2024.
[34] T. Fu *et al.*, "Multi-agent memory from a computer architecture perspective," arXiv preprint arXiv:2603.10062, 2026.
[35] N. Krishnan, "Advancing multi-agent systems through model context protocol," arXiv preprint arXiv:2504.21030, 2025.